RAMDEOBABA UNIVERSITY

Computer Science and Engineering

PROJECT SYNOPSIS

Session: 2026-27 | Semester:5th | Programme: IDEA Lab

POLYBRIDGE — A SCOPED MULTI-LANGUAGE TRANSPILER

Submitted by:

Saumya Khobragade — Roll No.: 56
Ayush Dutta — Roll No.: 22
Krish Parothi — Roll No.: 15
Hardik Sharma — Roll No.: 47

Under the Guidance of:

Dr. Padma Adane

Date:

19 / 09 / 2026

1. Background and Motivation

Modern software development uses multiple programming languages with different syntax and conventions. Translating a small algorithm between languages is often manual even when its logic is unchanged. Compiler design addresses this through structured parsing, intermediate representations and target-code generation [1]. Tree-sitter provides reusable parsing infrastructure for programming-language tooling [2]. PolyBridge applies these principles to a bounded educational problem: translating a procedural subset of Python, JavaScript, Java and Go through a shared IR, followed by executable output validation. The project targets compiler education, cross-language learning, code-generation experimentation and small deterministic translation tasks.

2. Problem Statement

Existing approaches include language-pair tools, syntax transformations and AI-assisted rewriting. For education, there is value in an inspectable pipeline that shows how source code is parsed, mapped to a common representation, regenerated in another language and validated. PolyBridge defines a controlled subset of Python, JavaScript, Java and Go and represents variables, arrays, assignments, expressions, loops and output in a common IR. Source and translated programs are executed with the same inputs and their outputs are compared.

3. Objectives

3.1 Build Tree-sitter based parsers for Python, JavaScript, Java and Go and obtain concrete syntax trees.
3.2 Design a language-agnostic IR for variables, arrays, assignments, expressions, loops and output.
3.3 Implement deterministic emitters for Python, JavaScript, Java and Go.
3.4 Develop an execution-based validation harness that compares the outputs of the source and translated programs.
3.5 Enforce the supported subset: source programs of at most 100 lines, with no external libraries and no user-defined classes or inheritance.
3.6 Keep the architecture extensible for additional languages, static analysis and future hardware/HDL back ends.

4. Literature Review and Research Gap

Sr.No.	Author(s) & Year	Title / Focus	Methodology / Key Contribution	Limitation / Gap
1	Aho et al., 2006	Compiler / IR	Parsing, IR and code-generation stages.	General framework.
2	Tree-sitter docs	Parsing / CST	Grammars and concrete syntax trees.	No common IR.
3	Wagner & Graham, 1998	Incremental parsing	Structured representations for language tools.	Not cross-language generation.
4	Padioleau et al., 2008	Coccinelle	Structured program matching and transformation.	Primarily C.
5	Lattner et al., 2020	MLIR	Reusable multi-level IR.	Broader infrastructure.
6	Lahti et al., 2019	High-level synthesis	Survey of abstraction-level trade-offs in HLS.	Hardware domain; not source-to-source.

Compiler literature establishes parsing, syntax-directed translation, IR generation and code generation as connected stages [1]. Tree-sitter supplies concrete syntax trees for language tooling [2]. Wagner and Graham present efficient incremental parsing techniques and the structured tree representations that support them [3]. Coccinelle demonstrates structured source transformation [4]. MLIR demonstrates reusable multi-level IR infrastructure shared across compilers and domain-specific frameworks, including hardware [5]. HLS research shows abstraction-level trade-offs in hardware design [6]. Parser-generator frameworks such as ANTLR [7] provide an alternative front-end route, and the formal grammar and automata theory underlying all such tools is set out by Hopcroft et al. [8]. These works support PolyBridge; the proposed project narrows the problem to an educational four-language pipeline with a custom IR, deterministic emitters and executable validation.

Research Gap: Existing work provides mature parsing, transformation and IR techniques; PolyBridge combines them into a small, inspectable four-language educational pipeline with behavioural validation.

5. Proposed Methodology / Plan of Work

The pipeline has five stages: parse source with a Tree-sitter grammar into a Concrete Syntax Tree; traverse the CST and convert supported nodes into a custom language-agnostic IR; emit Python, JavaScript, Java or Go using target-specific modules; execute source and generated programs with identical inputs; and compare standard outputs. Before comparison, a normalisation step accounts for language-level differences in numeric formatting and in integer versus floating-point division, so that behaviourally equivalent programs are not reported as mismatches. The IR covers variables, arrays, assignments, arithmetic/comparison expressions, for/while loops and output. Development starts with a Python-to-IR pipeline, followed by other emitters/parsers. Source programs use no external libraries and no user-defined classes; the emitters, however, generate the minimal boilerplate each target language requires in order to compile and run — a class wrapper and a main method for Java, and a package declaration with the standard fmt import for Go. Unsupported imports, libraries, classes, inheritance and other out-of-scope semantics are rejected with an explicit diagnostic message.

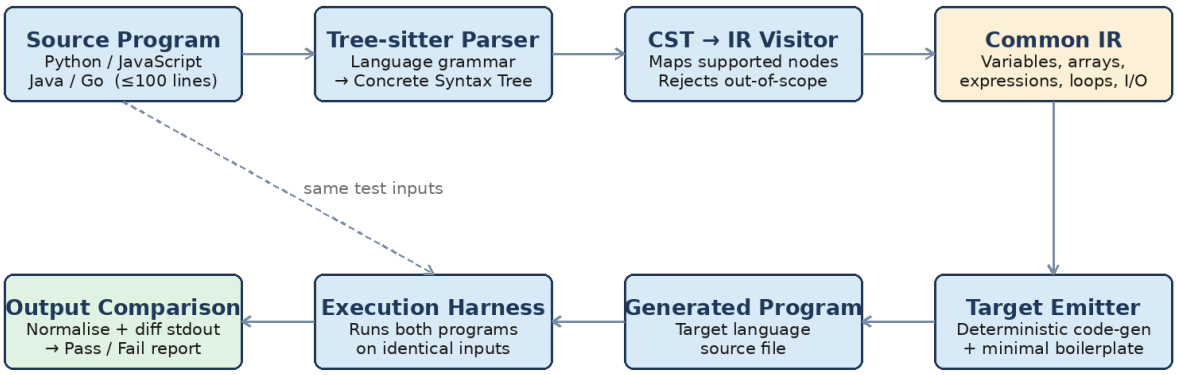


Fig. 1: PolyBridge translation and validation pipeline.

6. Technology, Tools and Platforms

Python, Tree-sitter, Python dataclasses, visitor-based CST-to-IR conversion, Python/JavaScript/Java/Go emitters, Git/GitHub, CLI, subprocess validation and local language runtimes/compilers. Testing uses a curated suite of small procedural programs; no external dataset is required.

7. Expected Outcomes, Deliverables and Functional Specifications

Deliverables include the CLI transpiler, parser modules, custom IR, four emitters, validation harness, test suite and documentation. Supported functions include variables, arrays, assignments, expressions, loops and output within scope. Identical inputs are used for source and translated programs, and standard outputs are compared. Unsupported constructs are explicitly reported.

8. Project Scope

The system translates a procedural subset among Python, JavaScript, Java and Go, limited to 100 lines of source code, written without external libraries and without user-defined classes or inheritance; generated code contains only the boilerplate its target language requires to compile. Applications include compiler education, programming-language learning, classroom demonstrations, algorithm prototyping, small code conversion, syntax/IR experimentation and code-generation testing. It is not an unrestricted production migration tool. Future work may add richer constructs, languages, optimisation, static analysis and a hardware-aware IR for selected HDL targets such as Verilog/SystemVerilog; this is future scope only.

9. Project Timeline

Literature/requirements → IR and architecture → implementation → testing → documentation. Development begins with Python-to-IR, followed by additional emitters/parsers; evaluation focuses on construct-level correctness and cross-language output equivalence.

Phase	Duration	Milestone / Deliverable
Literature review & requirement analysis	[Week 1 – 2]	Scope and constructs
Design & methodology finalisation	[Week 3 – 4]	IR, architecture, test plan
Implementation/development	[Week 5 – 10]	Python; JS, Java, Go
Testing & evaluation	[Week 11 – 12]	Tests and validation
Documentation & final submission	[Week 13 – 14]	Report and demo

10. References

[1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, "Compilers: Principles, Techniques, and Tools," 2nd ed., Pearson, 2006.
[2] Tree-sitter, "Tree-sitter: An incremental parsing system for programming tools," Tree-sitter Documentation. [Online]. Available: https://tree-sitter.github.io/tree-sitter/. [Accessed: Sep. 19, 2026].
[3] T. A. Wagner and S. L. Graham, "Efficient and Flexible Incremental Parsing," ACM Trans. Program. Lang. Syst., vol. 20, no. 5, pp. 980–1013, 1998, doi: 10.1145/293677.293678.
[4] Y. Padioleau, J. Lawall, R. R. Hansen, and G. Muller, "Documenting and Automating Collateral Evolutions in Linux Device Drivers," in Proc. 3rd ACM SIGOPS/EuroSys European Conf. Computer Systems (EuroSys '08), Glasgow, U.K., 2008, pp. 247–260, doi: 10.1145/1352592.1352618.
[5] C. Lattner et al., "MLIR: A Compiler Infrastructure for the End of Moore's Law," arXiv:2002.11054, 2020.
[6] S. Lahti, P. Sjövall, J. Vanne, and T. D. Hämäläinen, "Are We There Yet? A Study on the State of High-Level Synthesis," IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst., vol. 38, no. 5, pp. 898–911, 2019, doi: 10.1109/TCAD.2018.2834439.
[7] T. Parr, "The Definitive ANTLR 4 Reference," 2nd ed., Pragmatic Bookshelf, 2013.
[8] J. E. Hopcroft, R. Motwani, and J. D. Ullman, "Introduction to Automata Theory, Languages, and Computation," 3rd ed., Pearson, 2006.

Group Member Details

Roll No.	Name of Student	Signature of Student
56	Saumya Khobragade	
47	Hardik Sharma	
15	Krish Parothi	
22	Ayush Dutta	

Approved by:

Project Co- Coordinator

Project Coordinator

Associate Head of Department

Dean CSE